Mind Reading Using Machine Learning and EEG: Technical Report

Team KITKAT · Turgut Canberk Diner, Irem Ciloglu · Oct 18, 2026 · @irem

Summary

Our pipeline ranks the right video first 19.9% of the time and in the top 5 53.7% of the time on the sub-01..04 test sets (chance: 1.0% / 4.9%; organisers' baseline: about 4% / 11.7%).

It is an **encoding model**: it predicts the EEG that each candidate video should cause, then compares every prediction with the real recording and ranks the candidates by how well they match. The steps are:

1. Describe each video with four feature blocks from pretrained models and simple measurements: CLIP (what is shown), VideoMAE (motion), AST (sound) and low-level time courses (brightness, cuts, loudness).
2. Clean the EEG: downsample to 100 Hz, whiten the 17 electrodes, z-score, keep 0–1 s after onset; also compute rhythm power over 0–3 s.
3. Map features to EEG with ridge regression and a contrastive neural network, in an ensemble of subject, pooled and grand-average encoders.
4. Score each (recording, candidate) pair by Pearson correlation, correct for "hub" videos with column normalisation, and sort.

Every setting was chosen on held-out *training* videos. The test sets of sub-01..04 were used only to report the numbers above. On 5 fresh validation splits that were never used for any choice, the estimate is 8.4% top-1 / 27.0% top-5; that task is harder (about 150 candidates and single, noisier viewings).

Data and task

Six subjects watched 3-second videos with sound while 17 EEG electrodes recorded at 500 Hz, so each recording is an array of 17 × 1,500 samples. The data come from the EEG Moments Dataset (Gifford et al., 2026), recorded with 128 channels; the challenge provides 17 of them.

File	Content	Loaded by
`trn/sub-XX_trn.npz`	dict: video id → EEG (17, 1500); about 950 training videos per subject, ids 1–1000	`data.load_train` (sorted by id)
`tst/sub-XX_tst.npy`	array (102, 17, 1500), rows in shuffled order	`data.load_test`
`tst_stim/sub-XX_tst_stim.npy`	true video id for each test row; sub-01..04 only	`data.load_test` (labels, or None)
`stimuli_train/0001.mp4` … `1000.mp4`	training videos	`data.video_path`
`stimuli_test/1001.mp4` … `1102.mp4`	the 102 candidate videos	`data.video_path`

For every test recording we must output all 102 candidate ids ordered from most to least likely. The candidate videos never appear in training, so the model must learn a general rule from video content to brain response, not memorise videos. Scoring is top-1 and top-5 accuracy; we also track the median rank of the true video.

Test recordings are averages over more viewings (about 24) than training recordings (about 6). They are therefore about 35% quieter, with a std of about 0.32 against 0.49 for sub-01. This is why every file is normalised with its own statistics (see EEG preprocessing).

Rules that shaped the design:

- No test data may be used for training. sub-01..04 test labels are for validation and model selection only; we used them only for reporting.
- Unsupervised normalisation of a test recording, and of the video features, is allowed. Whitening and z-scoring per file rely on this.
- The deliverable is a pipeline that ranks candidates *given an EEG recording*. So column normalisation uses training recordings only and ranks one test recording at a time.

Code map and how to run

All logic lives in the `mindreading/` package. The notebooks only call it in order, so read the package files in the order of this table.

File	What it does	Main names to look for
`config.py`	Paths and constants: `DATA_DIR` (env `MINDREADING_DATA`, else `data/`), `cache/`, `weights/`, `submission/`, `results/`; subjects; train ids 1–1000, test ids 1001–1102; `SEED = 0`	`check_data_dir`
`data.py`	Reads the organisers' files	`load_train`, `load_test`, `video_path`
`video_features.py`	CLIP ViT-B/32 on 8 evenly spaced frames per video	`extract_clip`, `read_frames`
`features.py`	Low-level time courses, VideoMAE, AST, plus extractors we tried and dropped (sliding windows, dense CLIP)	`lowlevel_one`, `extract_videomae`, `extract_ast`, `REVISIONS`
`preprocess.py`	EEG cleaning and rhythm power	`prepare`, `whiten_channels`, `band_power`
`model.py`	Feature blocks, ridge encoder, ensemble, scoring, column normalisation, validation, weight export	`BLOCKS`, `Config`, `Encoder`, `Model`, `validate`, `dev_test`, `predict_scores`
`encoders.py`	Contrastive neural-network encoder (PyTorch)	`ContrastiveEncoder`, `SeedAverage`
`ranking.py`	Score matrix → ranked ids, format checks, submission file	`scores_to_ranked_ids`, `check_ranked`, `save_submission`

Other folders:

- `notebooks/01_…` and `02_…` are the organisers' baseline, kept for comparison.
- `notebooks/03_video_features` extracts and caches all video features in `cache/*.npz` (clip_b32, videomae, ast, lowlevel). This is the slow step; later runs skip any file that already exists.
- `notebooks/04_train_and_validate` runs `validate` and `dev_test`, writes `results/dev_test_scores.json`, and exports `weights/`.
- `notebooks/05_predict_submission` loads the weights and writes `submission/sub-05_ranked_candidate_list.npy` and `sub-06_…`.
- `past_trials/` holds every experiment script (`exp_*.py`) and `experiment_log.md`, the record of what we tried and why we kept or dropped it.

To run everything from raw data to submission files, use Python 3.11:

```
pip install -r requirements.txt
python run_all.py
```

`run_all.py` executes notebooks 03 → 04 → 05 with `nbclient` and saves them with their outputs. If the data is not in `data/`, set `MINDREADING_DATA` to the folder that contains `trn/`, `tst/`, `tst_stim/`, `stimuli_train/` and `stimuli_test/`.

Video features

Each video becomes one feature vector made of four blocks; together they describe what is shown, how it moves, what it sounds like and when things change. Videos are decoded with PyAV (`av`); audio is resampled to 16 kHz mono and padded or cut to 3 s.

Block	Source	Used in the model	Weight
`clip`	CLIP ViT-B/32	mean of 8 frames, 512 values	1.0
`mae`	VideoMAE-base (Kinetics-400)	layers 4, 8, 12; PCA 128	1.0
`low`	no model	8 signals × 10 bins (0–1 s)	0.6
`ast4`	AST (AudioSet)	layer 4; PCA 128	0.5

How each block is extracted:

- **clip:** 8 evenly spaced frames, each embedded and L2-normalised; the 8 vectors are averaged and normalised again.

- **mae:** 16 frames over 0–2.95 s; the hidden states of layers 4, 8 and 12 are each mean-pooled over tokens (3 × 768).
- **low:** 8 signals binned at 10 Hz over 3 s (30 bins): brightness, contrast, saturation, motion (frame difference), edge energy, scene-cut strength (grey-level histogram change), loudness (RMS dB) and sound onsets (spectral flux). Only the first 10 bins (0–1 s, the EEG window) are used.
- **ast4:** the 3 s soundtrack; layers 4, 8 and 12 are mean-pooled (3 × 768), and only layer 4 is used.

How a block is turned into model input (`load_block` in `model.py`):

1. `StandardScaler` fitted on the **training** videos (ids ⩽ 1000) and applied to all 1,102 videos.
2. If the block has a PCA size, `PCA(128)` is fitted on the training videos and the result is rescaled to unit average std.
3. The block is divided by √(number of columns), so every block has the same total size whatever its width.
4. `VideoFeatures` multiplies each block by its weight from `Config.blocks` and concatenates them.

Fitting the scaler and PCA on all videos without labels would also be allowed (unsupervised feature normalisation), but fitting on training videos keeps validation honest. The block weights act like a separate ridge strength per block: a weight w is equivalent to dividing that block's regularisation by w^2.

The pretrained models are pinned to exact Hugging Face commits in `features.REVISIONS` and `video_features.CLIP_REVISION`, so a rerun downloads the same weights (see Libraries).

EEG preprocessing

Each file is processed on its own, with no labels, so the same code runs on training and test recordings. `preprocess.prepare` produces the **wave** view used by the main encoders:

1. **Downsample** 500 → 100 Hz with `scipy.signal.decimate(…, 5, ftype="fir", zero_phase=True)`, an anti-aliasing FIR filter, as in the organisers' baseline. The shape goes from (n, 17, 1500) to (n, 17, 300).

2. **Whiten the channels** (`whiten_channels` , shrinkage 0.3). The 17 electrodes are about 0.78 correlated, mostly through noise they share. We take the file's own 17 × 17 channel covariance C, shrink it towards the identity, and multiply the EEG by C^(−1/2) (ZCA whitening). Afterwards the average correlation is about 0.12.
3. **Z-score per set** (`normalise(mode="set")`): each channel is centred and scaled over all recordings and time points of that file. This handles the quieter test files.
4. **Window** 0–1 s after onset (`window`): 100 time points. The result is flattened to 17 × 100 = 1,700 numbers per recording.

The shrinkage formula, with tr(C)/17 the average channel variance and I the identity:

- C_shrunk = (1 − s) · C + s · (tr(C) / 17) · I, with s = 0.3
- W = C_shrunk^(−1/2), and the whitened EEG is W · EEG

With s = 0 the covariance is inverted as is, which amplifies weak noisy directions (15.5% top-5 instead of about 21%). Values from 0.1 to 0.5 all perform the same within noise. Common average referencing (`Config.car`) is available but off; whitening replaced it.

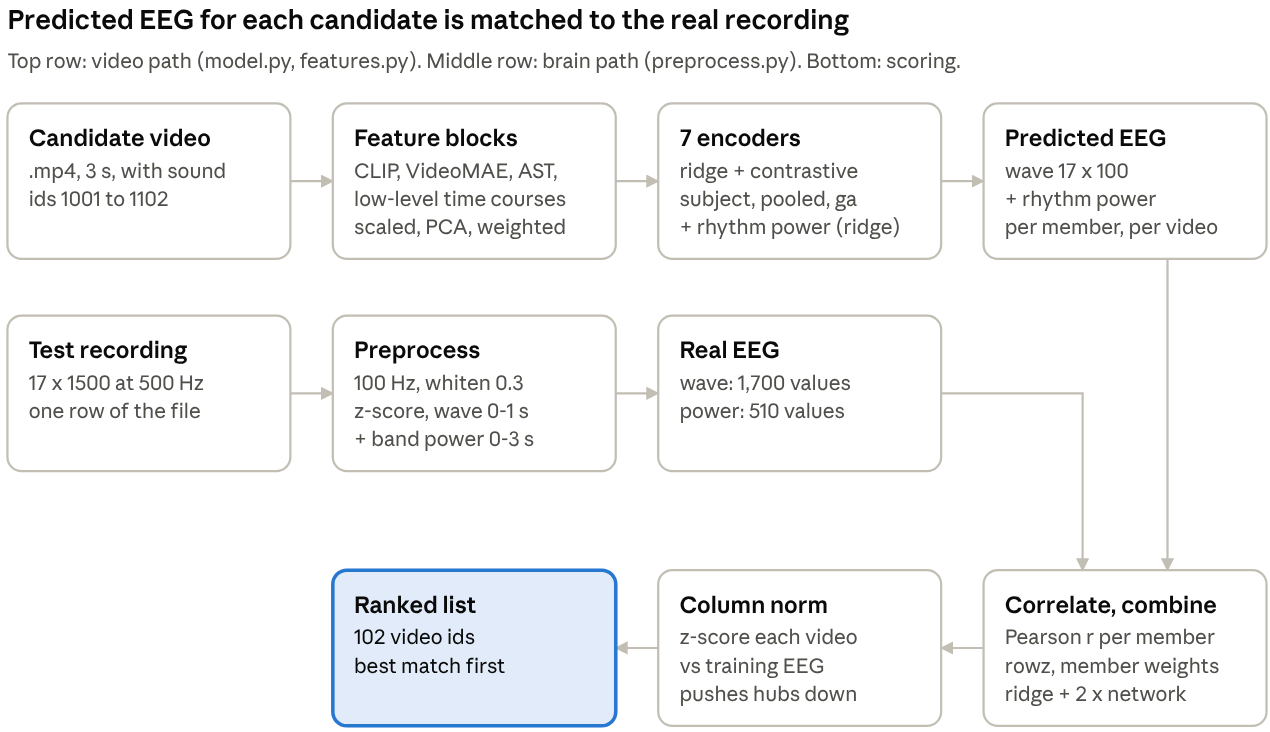
`preprocess.band_power` produces the **rhythm power** view, used by the power member:

1. Downsample, whiten (0.3) and z-score as above, but keep all 3 s.
2. Band-pass with a 4th-order Butterworth filter (`butter` + `sosfiltfilt` , zero-phase) into 5 bands: delta 1–4, theta 4–8, alpha 8–13, beta 13–30 and low gamma 30–45 Hz.
3. Square, average within 0.5 s bins (6 bins over 0–3 s) and take the log.
4. Z-score each of the 17 × 5 × 6 = 510 features within the file.

Why two views: the exact wave shape is predictable only in the first second (a 0–3 s wave window drops to 14.4% top-5). Slower changes in brain state, such as alpha dropping when a video is engaging, last all 3 s and are captured by power.

Model

The model is an ensemble of seven encoders per subject, each mapping a video's feature vector to a predicted EEG; their correlation scores are combined into one score per (recording, candidate) pair. Everything here is in `model.py` , except the network in `encoders.py` .



pipeline for one test recording · from model.py, preprocess.py, encoders.py

The same steps run for every one of the 102 candidates; only the bottom row compares them.

Ridge encoder (`Encoder`). `sklearn.linear_model.RidgeCV` with `alpha_per_target=True` and 21 strengths from 10^{-3} to 10^{7} (`ALPHAS`). Each of the 1,700 EEG outputs (electrode × time point) picks its own strength by efficient leave-one-out cross-validation. Features are mean-centred on the fit set.

Contrastive encoder (`ContrastiveEncoder`, PyTorch). It is trained for ranking rather than for copying the EEG:

- Architecture: a linear path plus a small MLP, `Linear(d, 1700) + [Dropout 0.2 → Linear(d, 1024) → GELU → Dropout 0.5 → Linear(1024, 1700)]`. The linear path starts from the ridge solution.
- Loss: symmetric InfoNCE over a batch of 256. The similarity is the correlation between real and predicted EEG, divided by a temperature of 0.05; recordings of the same video are not counted as negatives. A small MSE term (weight 0.1) is added.
- Training: AdamW (lr 1e-3, weight decay 5e-2), up to 150 epochs, early stopping (patience 15) on top-5 accuracy for 10% of the fit-set videos.
- `SeedAverage` trains 3 networks (seeds 0, 1, 2) and averages their predictions.

Ensemble members (`fit_subject`, `fit_shared`). Each member exists in a ridge form and a contrastive form (suffix `_c`):

Member	Trained on	Weight
`subject`	this subject's training recordings	1.0
`pooled`	all 6 subjects' training recordings together	0.25
`ga`	grand average: for each video, the EEG averaged over subjects	0.25
`power`	ridge only; this subject's rhythm-power features as the target	0.25

`pooled` and `ga` are shared by all subjects. They help because one subject's 950 noisy recordings are few, and they never use any test recording.

Scoring (`scores`). For each member, the score matrix (recordings × candidates) is the Pearson correlation between the real EEG and the member's prediction for each candidate (`correlation_matrix`). Rows are z-scored (`rowz`) before adding, so members with different spreads count by their weight only:

1. wave = Σ over members of weight · rowz(correlation scores)
2. part = rowz(wave) + 0.25 · rowz(power scores), computed once for ridge and once for contrastive
3. final score = rowz(ridge part) + 2 · rowz(contrastive part)

Column normalisation (`colnorm_stats`, `ranked_scores`; `Config.colnorm = True`). Some candidate videos produce a "typical" predicted response and correlate with almost every recording. These hubs crowd the top ranks. For each candidate we compute its mean and std of the final score over the subject's **training** recordings, and z-score its test score with them. A score then means "unusually high for this video". Only training data is used, and each test recording is ranked on its own.

Ranking (`predict_scores` → `ranking.scores_to_ranked_ids`). Each row of the 102 × 102 score matrix is sorted from high to low, and column indices are mapped to video ids 1001–1102.

Saved weights (`export_weights`). `weights/shared.joblib` holds the pooled and grand-average encoders plus the `Config`; `weights/sub-05.joblib` and `sub-06.joblib` hold each subject's own encoders. Arrays are stored as float16 with joblib compression so each

file stays under 100 MB (49 MB and 25 MB).

The `Config` dataclass at the top of `model.py` holds every setting above. It is the one place to change the model.

Validation protocol and rule compliance

Every design choice was made on held-out training videos; the sub-01..04 test sets were scored only to report progress, never to pick a setting.

- **Held-out training videos** (`validate`, `split_train_videos`). A random 15% of training video ids (150 videos) is hidden from every subject at once, so the shared pooled and grand-average encoders never see them either. The model is fitted on the rest, and each subject's hidden recordings are ranked among the hidden videos. This is repeated for seeds 0, 1 and 2, and the results are averaged over splits and all 6 subjects.
- **Fresh splits** (seeds 3–7). These five splits were never used for a decision. They give the unbiased estimate of 8.4% top-1 / 27.0% top-5 and protect against overfitting the three working splits.
- **Shuffled-label control** (`Config.shuffle_labels = True`). Videos are randomly reassigned to recordings before training. The pipeline then scores 1.0% / 4.9%, exactly chance, so nothing in the code leaks the answer.
- **Decision rule.** A change was kept only if it improved the score on every split, not just on average. Gains of under about 1 point top-5 are within split-to-split noise.
- **Column normalisation inside validation** uses only the fit-set recordings (`fit_ids`), just as the final model uses only training recordings.

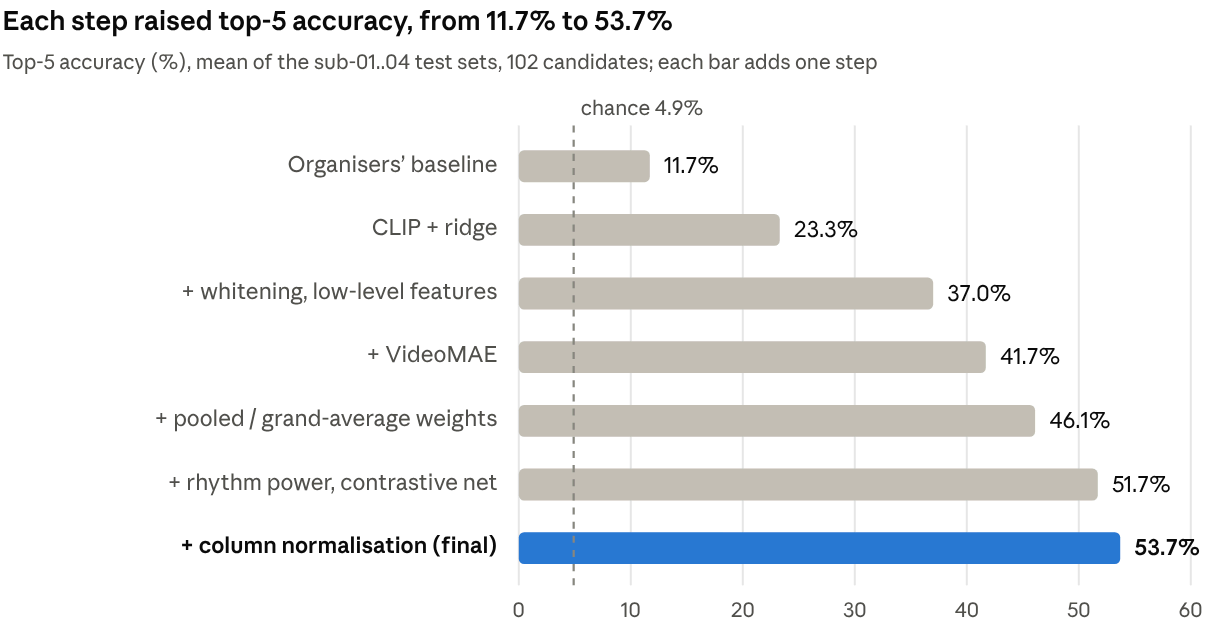
How the rules are met:

Rule	How the code meets it
No test data for training	Encoders, scalers and PCA are fitted on training recordings and training videos only
sub-01..04 test labels for validation only	`dev_test` reports them; no setting was chosen from them
Unsupervised normalisation of test recordings allowed	Whitening, z-scoring and power z-scoring use each test file's own statistics, without labels

Rule	How the code meets it
Rank given an EEG recording	Column normalisation uses training-recording statistics, so each test recording is ranked independently of the others

Results

The final model reaches 19.9% top-1, 53.7% top-5 and a median rank of 5.0 on the sub-01..04 test sets; the biggest single jumps came from CLIP features and from whitening.



experiment_log.md milestones · sub-01..04 test sets, top-5 · reported only, not used for choices

Each bar is one test run, so steps of about 2 points are within noise. For example, column normalisation added only +0.7 top-5 on the fresh validation splits; 53.7% is the final model's score, not its gain.

Per subject on the test sets (102 candidates):

Subject	Top-1 (%)	Top-5 (%)
sub-01	25.5	60.8
sub-02	14.7	46.1

Subject	Top-1 (%)	Top-5 (%)
sub-03	11.8	52.0
sub-04	27.5	55.9

On the 5 fresh validation splits (about 150 candidates, single noisier viewings) the model scores 8.4% top-1 / 27.0% top-5. With shuffled labels it falls to 1.0% / 4.9%, which is chance. The full model's scores are in `results/dev_test_scores.json`.

Experiments that did not help

Most ideas we tried did not beat the current settings on every split, so they are off or left at their defaults. Each one has a script in `past_trials/` and an entry in `experiment_log.md`.

Idea	Result (validation)	Decision
Whitening shrinkage 0.1–0.5	20.2–21.3% top-5, flat; 0.0 gives 15.5%	kept 0.3
Covariance from file / training set / noise only	20.9 / 21.0 / 20.9% top-5	kept the file's own
Banded ridge (strength per block)	worse on every split: 26.0 → 25.4% top-5	kept off
AST layer 8, 12 or all instead of 4	audio alone 9.5 / 9.8 / 9.8 / 10.7% top-5	kept layer 4
PCA size other than 128	128 best	kept 128
EEG sampling rate 25–500 Hz	all within noise	kept 100 Hz
Wave window 0–3 s instead of 0–1 s	14.4% top-5	kept 0–1 s
CLIP ViT-L/14	middle layers 5.0% vs 3.5% top-1 for its final embedding	kept B/32
Decoding (EEG → video features)	chance	kept encoding
Network trained to copy the EEG (MSE only)	23.4% vs 28.3% top-5 with InfoNCE	kept InfoNCE

Notes: banded ridge's learned scales stayed near 1 (clip ×0.92, ast4 ×0.72, low ×0.94, mae ×1.18), and its inner split costs 20% of the training data. The scripts are `exp_whiten.py`, `exp_noisecov.py`, `exp_banded.py` and `exp_ast_layers.py`.

For gains, the contrastive network added 2.9 points top-5 on the fresh splits (3.6 points on splits 0–2, 24.7% → 28.3%, which were also used to tune it). Column normalisation added 2.2 points top-5 before the power member existed. On the final model it adds +0.4 / +1.4 / +1.2 points top-1 on the three splits, with top-5 flat, because rhythm power had already absorbed part of the hub effect.

Failure cases and limitations

The model works well on average but varies a lot between subjects, and about half of the test recordings still miss the top 5.

- **Subjects differ.** Top-1 ranges from 11.8% (sub-03) to 27.5% (sub-04) on the test sets. We have not found why: it may be signal quality, attention or electrode contact. The shared pooled and grand-average members soften this but do not remove it.
- **Only the first second is predictable as a wave.** After 1 s the exact EEG shape varies too much between viewings, so the model uses only rhythm power for 1–3 s, and that view adds just 0.25 weight.
- **Similar videos get confused.** The features describe content, motion and sound; two videos with the same kind of scene produce similar predicted EEG. A per-video analysis of which test videos are never found has not been done yet.
- **Test and validation conditions differ.** Test recordings are averaged over about 24 viewings and are ranked among 102 candidates; validation recordings come from about 6 viewings among about 150 candidates. That is why test scores (53.7% top-5) are much higher than the fresh-split estimate (27.0%). sub-05 and sub-06 should land near the sub-01..04 test numbers, but with the same spread between subjects.
- **Linear core.** Ridge is linear in the features, and the network adds only a small non-linear path. Gains rely on the quality of the pretrained features, which were trained on images, actions and sounds, not on brains.
- **Cost.** Feature extraction takes the most time (four pretrained models over 1,102 videos on CPU). Training the 3-seed contrastive networks for 6 subjects plus the shared members is the second-largest cost.

Brain vs AI

Features from AI models trained on images, actions and sounds predict EEG well enough to identify videos, but our results also show where brains and these models differ.

	What we observed	Evidence
Similar	AI features predict brain responses	53.7% top-5, 4.6× the organisers' baseline
Similar	Middle vision layers fit the EEG better than the final output, like mid-level visual areas	CLIP-L/14 intermediate layers 5.0% against 3.5% top-1 for its final embedding
Similar	Motion and change drive both	Banded ridge gave the motion block (VideoMAE) the largest scale, ×1.18; low-level motion and cut features help
Different	A stronger AI model is not more brain-like	CLIP-L/14, a much better object recogniser, did not beat B/32 for EEG
Different	The audio model's layer hierarchy is not visible in EEG	AST layers 4, 8 and 12 all score about 10% top-5 alone
Different	The brain has a slow state over seconds	Rhythm power over 0–3 s adds 3.8 points top-5; frame-based models have no such state
Different	Brains vary between viewings and people; models are deterministic	Test recordings from 24 viewings are far cleaner than training ones from 6; top-1 ranges 11.8–27.5% across subjects

Caveat: with 17 scalp electrodes we see only coarse, fast activity. Finer object meaning may be present in the brain but invisible to EEG.

Libraries, versions and pretrained models

The pipeline runs on Python 3.11 with the exact versions pinned in `requirements.txt`; PyTorch is the CPU build by default.

Library	Version	Used for
torch	2.14.1 (CPU index)	running the pretrained models; the contrastive network
transformers	5.20.1	`CLIPModel`, `VideoMAEModel`, `ASTModel` and their processors
av (PyAV)	18.1.0	decoding video frames and audio from the .mp4 files
numpy	2.4.6	all array work; .npz caches and .npy submissions
scipy	1.17.1	`decimate` (downsampling), `butter` + `sosfiltfilt` (band power)
scikit-learn	1.9.1	`RidgeCV`, `StandardScaler`, `PCA`
joblib	1.6.0	saving the trained weights
pandas	3.0.6	result tables in the notebooks
matplotlib	3.11.2	figures in the notebooks
pillow	12.3.0	frame images passed to the CLIP processor
nbformat, nbclient	5.11.1, 0.11.0	`run_all.py` executes the notebooks
ipykernel, jupyter	7.4.0, 1.1.1	notebook kernel

Pretrained models (public, downloaded from Hugging Face at the pinned commit; none are fine-tuned by us):

Model	Hugging Face id	Trained on
CLIP ViT-B/32	openai/clip-vit-base-patch32	image–text pairs
VideoMAE-base	MCG-NJU/videomae-base-finetuned-kinetics	Kinetics-400 actions
AST	MIT/ast-finetuned-audioset-10-10-0.4593	AudioSet sounds

Pinned revisions (`features.REVISIONS`):

```
clip_b32   3d74acf9a28c67741b2f4f2ea7635f0aaf6f0268
videomae   488eb9a0565f257b32866000305c8178965eb9f6
ast        f826b80d28226b62986cc218e5cec390b1096902
```

Submission format and reproducibility

The submission is two files, `submission/sub-05_ranked_candidate_list.npy` and `sub-06_ranked_candidate_list.npy`, each an int64 array of shape (102, 102).

- Row i belongs to row i of `tst/sub-XX_tst.npy`, in the file's original order (never re-sorted).
- Each row lists all 102 candidate **video ids** (1001–1102, not column indices), from most to least likely.
- `ranking.check_ranked` rejects a wrong shape, a float dtype, column indices or a row that is not a permutation of the ids. `save_submission` calls it before writing. Running `python -m mindreading.ranking` self-tests this on synthetic data (perfect scores must give 100%, random scores near chance).

Reproducing the results:

- All randomness is seeded from `config.SEED = 0`: validation splits, PCA, the early-stopping split and the network seeds (0, 1, 2).
- Model weights are pinned by Hugging Face revision, and library versions by `requirements.txt`.
- Video features are cached in `cache/*.npz`; preprocessed EEG is computed on the fly. Deleting `cache/` forces a full re-extraction.
- `weights/` contains the trained models, so notebook 05 can rebuild the submission without retraining.
- Small differences may remain across machines: PyTorch training on CPU versus GPU is not bit-identical, and the networks are stored as float16.

References

1. Gifford, Oyarzo, Zonneveld, Sartzetaki, Groen and Cichy (2026). A large dataset of human EEG responses to short naturalistic videos for studying dynamic visual event processing (EEG Moments Dataset). arXiv:2608.28768

2. Radford et al. (2021). Learning transferable visual models from natural language supervision (CLIP). arXiv:2103.00020
3. Tong et al. (2022). VideoMAE: masked autoencoders are data-efficient learners for self-supervised video pre-training. arXiv:2203.12602
4. Gong, Chung and Glass (2021). AST: Audio Spectrogram Transformer. arXiv:2104.01778
5. van den Oord, Li and Vinyals (2018). Representation learning with contrastive predictive coding (InfoNCE loss). arXiv:1807.03748
6. Nunez-Elizalde, Huth and Gallant (2019). Voxelwise encoding models with non-spherical multivariate normal priors (banded ridge). NeuroImage.
7. Guggenmos, Sterzer and Cichy (2018). Multivariate pattern analysis for MEG: a comparison of dissimilarity measures (noise normalisation by whitening). NeuroImage.
8. Radovanović, Nanopoulos and Ivanović (2010). Hubs in space: popular nearest neighbors in high-dimensional data (hubness). Journal of Machine Learning Research.
9. Hugging Face model pages for the pinned checkpoints are linked under Libraries.

Reference 1 was checked on arXiv; references 2–8 are cited from memory and were not re-opened for this report.